



OXETECH Academy



SECTI
Secretaria de Estado da
Ciência, Tecnologia
e da Inovação



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

Cronograma



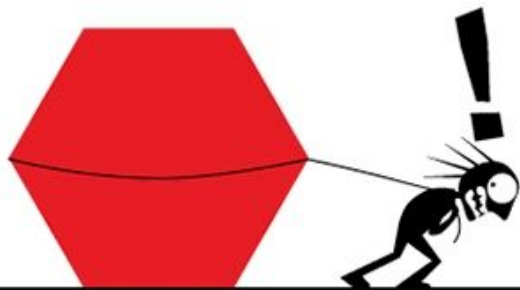
Módulos	Conteúdo
Módulo 1 — Fundamentos da Engenharia Moderna	Processos, versionamento e code review Gestão de dependências, Documentação viva
Módulo 2 — Clean Code e SOLID	Legibilidade, funções e acoplamento, SOLID aplicado, Refatoração incremental
Módulo 3 — Back -end com Node.js	Factory, Strategy, Observer, Repository, Adapter, Decorator, Anti -patterns comuns

Módulos	Conteúdo
Módulo 4 — Arquiteturas e Escalabilidade	Monólito modular, hexagonal, Microserviços e mensageria, DDD básico
Módulo 5 — Qualidade, Testes e Segurança	Pirâmide de testes e cobertura, Sonar, linters, quality gates, Segurança básica em apps
Módulo 6 — CI/ CD e Operação	Pipelines (build/ test/ deploy), Blue green, canary e rollback, Observabilidade (logs/ metrics/ traces)
Módulo 7 — Projeto Final Integrador	Sistema modular com integração contínua, Qualidade e observabilidade integradas, Defesa técnica

Entregas - Cascata vs Agil



THE WATERFALL PROCESS



Este projeto é muito grande!
E se precisarmos mudar algumas necessidades?

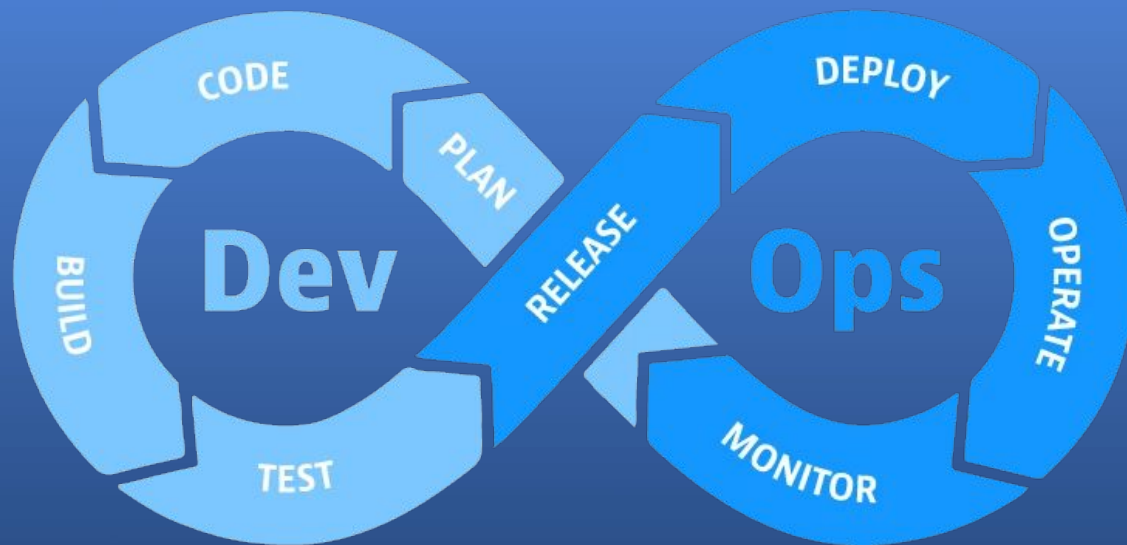
THE AGILE PROCESS



Vamos dividir o projeto em pequenas partes!
Podemos realizar mudanças a qualquer tempo.

Devops





Integração Contínua, Deploy Contínuo e Observabilidade



No cenário atual de desenvolvimento de software, a velocidade e a confiabilidade na entrega de novas funcionalidades são cruciais para o sucesso. A Integração Contínua (CI) e o Deploy Contínuo (CD), juntamente com estratégias de deploy avançadas, gerenciamento de release, Shift-Right Testing e Observabilidade, formam a espinha dorsal de um processo de entrega de software moderno e eficiente.

Integração Contínua (CI)



A Integração Contínua (CI) é uma prática de desenvolvimento de software onde os desenvolvedores integram seu código em um repositório compartilhado várias vezes ao dia. Cada integração é então verificada por um build automatizado (incluindo testes automatizados) para detectar erros de integração o mais rápido possível

Princípios Fundamentais da CI



Controle de Versão	Todo o código-fonte, scripts de build, testes e configurações devem estar sob controle de versão (e.g., Git). Isso garante rastreabilidade e a capacidade de reverter mudanças [1].
Automação do Build	O processo de compilação do código deve ser totalmente automatizado, garantindo que o software possa ser construído de forma consistente a qualquer momento [1].
Testes Automatizados:	Uma suíte robusta de testes automatizados (unitários, de integração) deve ser executada a cada integração para validar a funcionalidade e a qualidade do código. Isso permite feedback rápido sobre a introdução de defeitos [1].

Feedback Rápido	Os desenvolvedores devem receber feedback imediato sobre o sucesso ou falha de suas integrações e builds. Isso permite que os problemas sejam corrigidos
Qualidade de Código	Ferramentas de análise estática de código e linters devem ser integradas ao pipeline de CI para garantir que o código siga padrões de qualidade e segurança, identificando vulnerabilidades e problemas de estilo antes que se tornem mais difíceis de resolver [4].

Melhores Práticas para Construção de um CI



Commit Frequente	Desenvolvedores devem fazer commits pequenos e frequentes para o repositório principal. Isso minimiza conflitos de merge e mantém o código em um estado sempre integrável
Otimizar Estágios do Pipeline:	Garanta que cada estágio do pipeline de CI seja o mais rápido e eficiente possível. Testes unitários devem ser executados primeiro, seguidos por testes de integração e, por fim, testes end-to-end, que são mais lentos
Construir Artefatos Apenas Uma Vez	O artefato de build deve ser construído uma única vez e promovido através dos ambientes. Isso garante que o que é testado é exatamente o que será implantado [2].

Automação de Testes Abrangente	Invista em uma pirâmide de testes robusta, com uma base ampla de testes unitários, uma camada intermediária de testes de integração e um número menor de testes end-to-end críticos
Qualidade de Código e Análise de Segurança	Integre ferramentas de análise estática de código (SAST) e scanners de dependências no pipeline para identificar vulnerabilidades e problemas de qualidade no início do ciclo de desenvolvimento [4].

Estratégias de Integração Contínua



Estratégia	Complexidade	Frequência de Deploy	Risco
Trunk Based Development	Baixa	Alta	Baixo
GitHub Flow	Baixa	Alta	Baixo
Git Flow	Alta	Média	Médio
Branch by Abstraction	Alta	Alta	Baixo
Feature Flags	Média	Muito Alta	Muito Baixo

O que é um Pipeline?



Um pipeline é um conjunto de etapas automatizadas que transforma o código-fonte em um software pronto para produção.

Objetivos

- Automatizar tarefas repetitivas;
- Garantir qualidade;
- Reduzir erros humanos;
- Fornecer feedback rápido;
- Aumentar a frequência de deploy.

Testes Automatizados



Por que Automatizar Testes?

Em um processo de Integração Contínua, cada alteração de código deve ser validada rapidamente.

Sem testes automatizados:

- Maior risco de defeitos
- Feedback lento
- Deploys inseguros
- Alto custo de manutenção
- Baixa frequência de entrega

Qualidade e Segurança em Pipelines de CI



Por que Qualidade e Segurança são Importantes?

No modelo tradicional, qualidade e segurança eram verificadas apenas no final do projeto.

Problemas:

1. Bugs encontrados tarde.
2. Alto custo de correção.
3. Vulnerabilidades em produção.
4. Atrasos nas entregas.
5. Riscos financeiros e reputacionais.